AnyStream — App Plan

Oct 28, 2026 · @Will

Overview and goals

AnyStream is one React Native app for Vega OS (next-gen Fire TV) that feels like Netflix but plays from your own Jellyfin and Plex servers. Each server is browsed on its own: Home, libraries and Continue Watching show only the active server. Search is the one place that spans every signed-in server, and when a title exists on more than one, AnyStream offers to switch to it. It ships to the Amazon Appstore, so the plan is held to store quality.

v1 goals

- Sign in to Jellyfin (username/password or Quick Connect) and Plex (PIN link at plex.tv/link), multiple servers at once
- Netflix-style home: hero banner, horizontal rows, smooth D-pad focus with zoom-on-focus posters
- Reliable playback: direct play where the TV can decode it, server transcode to HLS where it can't
- Resume points, watched state and progress synced back to the server
- Multiple profiles (Jellyfin users / Plex Home users) with a "Who's watching?" screen

Out of scope for v1: live TV/DVR, music, offline downloads, casting.

Test target: your own Jellyfin at watch.whtvmovies.com (HTTPS via Caddy) is ideal — it exercises the remote, TLS and transcoding paths a real user hits.

Platform: Vega OS constraints

Vega apps are React Native, not web React — you write `View` / `Text` / `Image`, not DOM, and must use Amazon's `@amazon-devices/*` ports of native libraries. (RN for Vega overview)

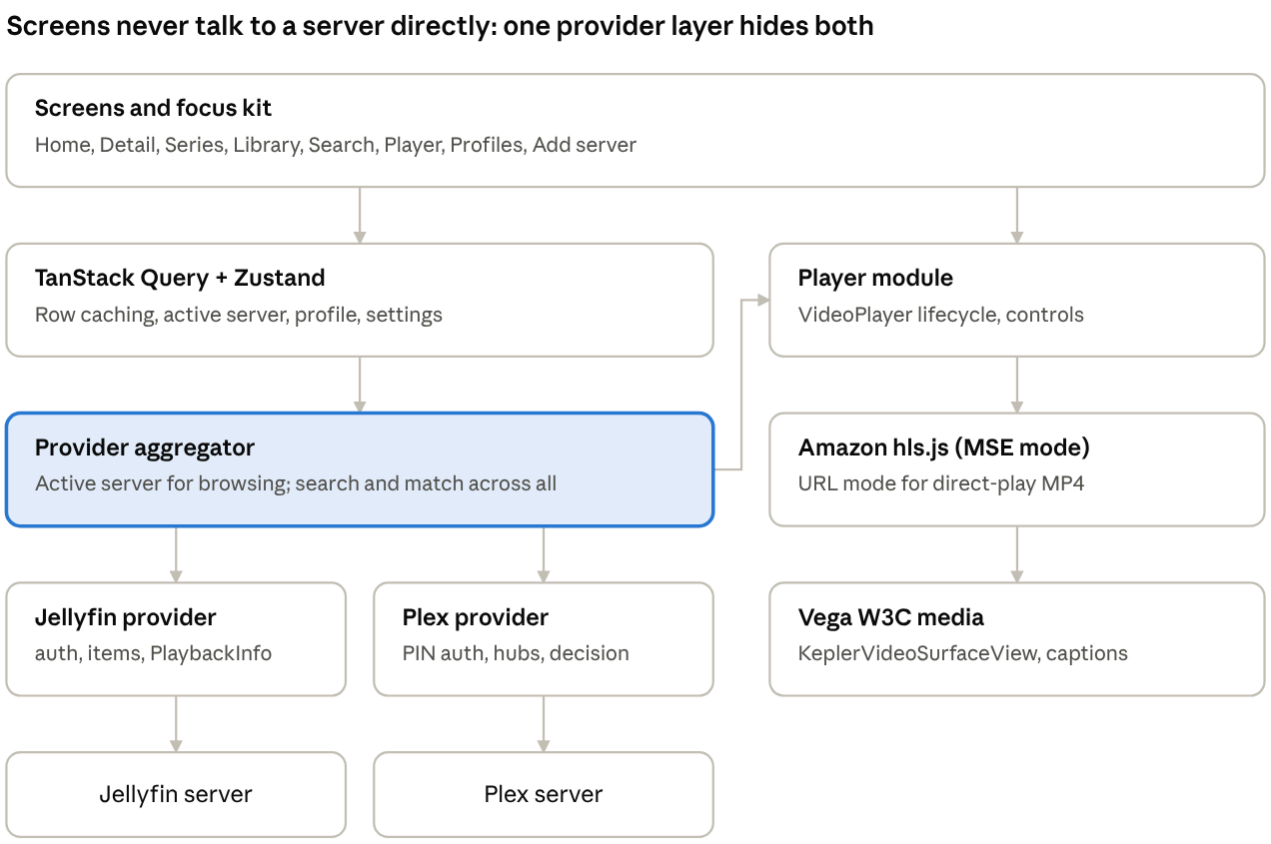
Area	What Vega gives you	Implication for this app
Runtime	RN 0.72 and RN 0.83, New Architecture (Fabric, TurboModules), runtime shipped in the OS	Target RN 0.83 for new work; keep native deps to the ported list

Area	What Vega gives you	Implication for this app
Focus	Cartesian D-pad focus by default, `TVFocusGuideView`, `nextFocus*` props, imperative `FocusManager.setNextFocus()`	Build a small focus kit early; rows and hero need explicit focus traps (focus docs)
Media	W3C media APIs (`VideoPlayer`, `KeplerVideoSurfaceView`, `KeplerCaptionsView`) from `@amazon-devices/react-native-w3cmedia`; URL mode for MP4, MSE mode for HLS/DASH	Direct play via URL mode; transcodes via HLS in MSE mode (playback modes)
JS players	Amazon-patched Shaka Player, hls.js, dash.js	Use Amazon's hls.js (Jellyfin and Plex both transcode to HLS)
Libraries	react-navigation, FlashList, fast-image, MMKV, netinfo, svg, linear-gradient, Reanimated ports	Covers navigation, virtualised rows, image caching and storage (supported libraries)
Tooling	Vega Studio (VS Code extension), Vega CLI, Vega Virtual Device simulator, VDA device installer	Develop on the simulator; profile on a real Fire TV before each milestone

Anything with native code that isn't on the supported list must be ported, so avoid pulling in arbitrary RN packages.

Architecture and server abstraction

Every screen talks to one `MediaProvider` interface, so adding Emby or another backend later is a new folder, not a rewrite.



app architecture · 4 layers, 2 servers

The aggregator hands the player a playback decision (stream URL plus mode); the player never knows which server it came from.

```
interface MediaProvider {
  kind: 'jellyfin' | 'plex';
  authenticate(): Promise<Session>;
  getHomeRows(profileId: string): Promise<Row[]>;
  getItem(id: string): Promise<Item>;
  getChildren(id: string): Promise<Item[]>;  // seasons, episodes
  search(query: string): Promise<Item[]>;
  getPlayback(id: string, profile: DeviceProfile): Promise<PlaybackDecision>;
  reportProgress(id: string, positionMs: number, state: 'playing' | 'paused'
| 'stopped'): Promise<void>;
  imageUrl(item: Item, kind: 'poster' | 'backdrop' | 'logo', width: number):
string;
}
```

Item IDs are namespaced (`jf:<serverId>:<itemId>` , `plex:<machineId>:<ratingKey>`) so search results from different servers never collide.

Server separation and matching. The aggregator holds one active server; every browse call goes only to that provider. Search fans out to all providers in parallel and groups hits that are the same title. Matching uses external IDs — Jellyfin `ProviderIds` and Plex `guid` entries (TMDB, IMDb, TVDB) — with title + year as a fallback, so the interface gains a `findByExternalIds(ids)` method. The same lookup powers the "Also on" button on title detail; switching servers clears the browse cache and keeps the search results.

Screens and UX

Nine screens cover v1; the Home screen and Player carry most of the work.

Screen	Contents	Key focus behaviour
Add server	Choose Jellyfin or Plex, enter URL (Jellyfin) or show PIN + QR for plex.tv/link	QR via react-native-qrcode-svg; phone does the typing
Who's watching	Profile tiles from Jellyfin users / Plex Home users, optional PIN	Focus starts on last-used profile
Home	Hero banner (backdrop, logo, overview, Play / More Info), then rows: Continue Watching, Next Up, Recently Added per library, genre rows — all from the active server only	Up from first row returns to hero; left edge opens side nav; each row remembers its last focused index
Side nav	Server switcher at the top (active server's name and type), then Search, Home, Movies, TV Shows, Collections, Settings	Collapsed icons, expands on focus
Library	Horizontally scrolling genre menu (All, then each genre in the library, as pill chips), poster grid below, sort (A–Z, added, rating) and an unwatched filter. Genres come from Jellyfin `/Genres?ParentId=<library>` and Plex `/library/sections/<id>/genre`	Up from the grid's top row lands on the selected genre chip; Left/Right scrolls the chips; focusing a chip for 400 ms reloads the grid; the menu remembers the last genre per library
Title detail	Backdrop, metadata, cast, Play / Resume / Trailer, More Like This row, plus an "Also on <server>" button when another signed-in server has the same title; choosing it switches the active server and reopens the title there	Default focus on Resume if a resume point exists

Screen	Contents	Key focus behaviour
Series detail	Season tabs, episode list with thumbnails and progress bars	Left/Right switches season, Down enters episodes
Search	On-screen keyboard, results grid updating as you type (debounced 300 ms), searching every signed-in server; one tile per title, badged with each server that has it. Selecting a title on a non-active server asks "Switch to <server>?" before opening it	Keyboard on left, results on right
Player	Full-screen video, scrubber with trickplay thumbnails, audio/subtitle picker, Skip Intro, next-episode countdown	Controls auto-hide after 5 s; Back closes overlays before exiting

Netflix feel, concretely

- Focused poster scales to 1.1 with a 150 ms ease and a white focus ring; the hero backdrop crossfades to the focused title after 600 ms of dwell
- Images requested at the exact tile size from the server's image endpoint, cached with fast-image
- Skeleton tiles while rows load; no spinners on Home
- 10-foot rules: 48 px safe margins, minimum 24 px body text, high contrast on dark backgrounds

Playback

Let the server decide direct play vs transcode by sending it an honest device profile; the app then picks URL mode or MSE mode from the answer.

1. **Describe the TV.** Ship a static device profile: containers (MP4, MKV, TS), video codecs (H.264, HEVC, VP9, AV1 if the device reports it), audio (AAC, AC3, EAC3), max bitrate and resolution. Probe real support on device and trim the profile to match.
2. **Ask the server.**
   - Jellyfin: `POST /Items/{id}/PlaybackInfo` with the profile → returns a direct stream URL or a `TranscodingUrl` (HLS `master.m3u8`)
   - Plex: `GET /video/:/transcode/universal/decision` → direct play, direct stream (remux) or transcode, then `start.m3u8`

3. **Play it.** Direct-play MP4 → `VideoPlayer.src` (URL mode). Anything HLS → Amazon-patched hls.js in MSE mode on `KeplerVideoSurfaceView`.
4. **Report progress.** Jellyfin `Sessions/Playing`, `/Progress` every 10 s and `/Stopped`; Plex `/:/timeline` every 10 s and `/:/scrobble` at 90%. Without this, resume and "watched" break.
5. **Clean up.** On surface destroy: clear the surface handle, deinitialise the player, and tell the server to stop the transcode so it doesn't keep burning CPU.

Subtitles and audio: text subs (SRT, VTT) render in `KeplerCaptionsView`; image subs (PGS, VobSub) need the server to burn them in, which forces a transcode — flag this in the picker. Audio track changes on a transcode restart the stream at the current position.

Extras for v1.1: trickplay thumbnails (Jellyfin trickplay tiles, Plex BIF), Skip Intro (Jellyfin media segments, Plex intro markers), next-episode autoplay.

Tech stack and project structure

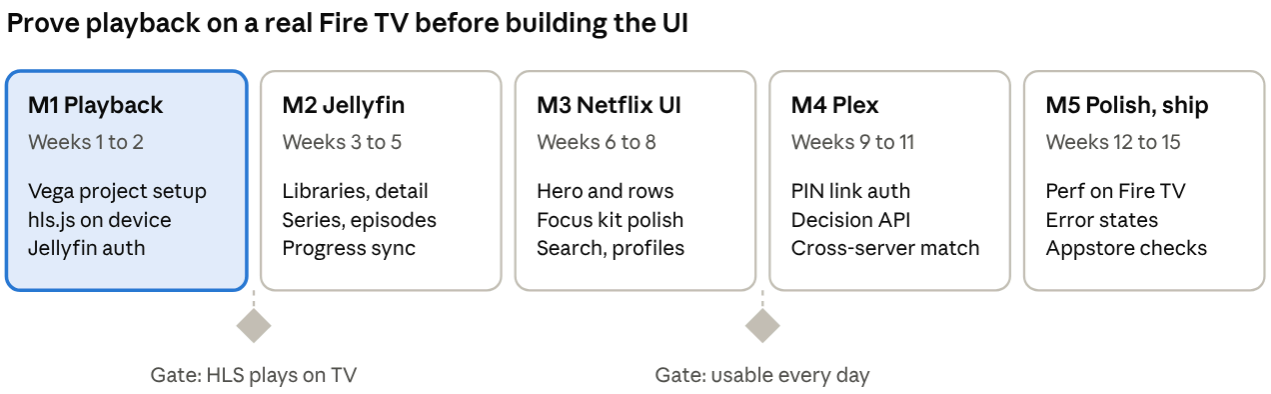
TypeScript throughout, with every native dependency taken from Amazon's ported list.

Concern	Choice	Why
Navigation	react-navigation native-stack (Vega port)	Supported; handles Back key
Lists and rows	FlashList (Vega port)	Recycling keeps 20+ rows of posters smooth
Server data	TanStack Query (pure JS)	Caching, stale-while-revalidate for rows, retries
App state	Zustand (pure JS)	Tiny; holds active server, profile, player state
Storage	MMKV (Vega port)	Tokens, server list, last profile
Images	react-native-fast-image (Vega port)	Disk caching (note: cache-control headers not honoured)
HTTP	fetch, thin wrapper per provider	Avoids extra deps; axios also tested on Vega
Player	`@amazon-devices/react-native-w3cmedia` + Amazon's hls.js	Only supported route to HLS
Tests	Jest + Reassure for render perf; Appium for end-to-end on device	All listed as working on Vega

```
src/
  providers/
    types.ts          // MediaProvider interface, Item, Row, PlaybackDecision
    jellyfin/         // auth, library, playback, progress
    plex/             // pin auth, library, decision, timeline
    aggregate.ts      // merges providers into unified rows
  player/             // VideoPlayer lifecycle, hls.js wrapper, controls
  focus/              // FocusRow, FocusGrid, useRememberedFocus
  screens/            // Home, Detail, Series, Library, Search, Player,
Profiles, AddServer, Settings
  components/         // Hero, PosterCard, Row, Skeleton, SideNav, Keyboard
  state/              // zustand stores
  theme/              // colours, spacing, type scale for 10-foot UI
```

Milestones

Roughly 15 weeks part-time, Jellyfin first because it's your own server and its API is fully documented; Plex follows once the provider interface has settled.



roadmap · 5 milestones, 2 gates

Week counts are estimates for one developer; the first gate is a hard stop — if HLS won't play smoothly on hardware, fix that before any UI work.

Release polish

A store release needs these met before submission; M5 is given four weeks for them.

Area	Standard for AnyStream
First run	Welcome screen, then add server with three paths: Jellyfin Quick Connect, Plex PIN with QR code, or manual URL. Discover servers on the local network if Vega allows UDP broadcast; otherwise manual entry only
Error and empty states	A designed screen with a retry action for: no network, server unreachable, expired sign-in, empty library, no search results, unplayable file. No raw error text
Performance targets	Home usable in under 2 s on warm start, rows scroll without dropped frames, direct play starts in under 3 s. Measured with Flashlight on a Fire TV Stick 4K Select
App lifecycle	Pause cleanly on Alexa interruptions, screensaver, HDMI disconnect and switching to Bluetooth audio; these are common Appstore failures (test guide)
Accessibility	`accessibilityLabel` and role on every focusable element, tested with VoiceView; respect system caption settings (Vega accessibility)

Area	Standard for AnyStream
Branding	AnyStream logo, app icon, splash and Appstore screenshots and background art. Jellyfin and Plex named only descriptively, never as app branding; check each project's trademark guidance
Privacy	Privacy policy URL; credentials and tokens stored only on the device; no analytics without consent; crash reporting with personal data stripped
Localisation	All strings externalised from day one; English at launch
Packaging	Manifest registers `com.amazon.category.main` or the app won't launch from the Fire TV home screen and fails submission; release VPKG uploaded through the Developer Console (submission)

Risks and open questions

The biggest risk is playback, so prove it on real hardware in the first milestone before building UI.

Risk	Impact	Mitigation
hls.js on Vega behaves differently from desktop	Stalls or no playback	Spike in M1 against your Jellyfin; fall back to Shaka (also patched by Amazon)
Device profile wrong	Needless transcodes, or direct play that fails	Probe codecs on device; on playback error, retry once forcing transcode
Plex API is semi-documented	Breaks on server updates	Isolate behind the provider interface; pin tested server versions
Poster rows drop frames	App feels cheap	FlashList, exact-size images, profile with Flashlight on a real Fire TV
Self-signed / plain-HTTP home servers	Connection refused	Clear error screen; document HTTPS setup (Caddy works)

Risk	Impact	Mitigation
Appstore review	Rejection for personal-server apps	Read Vega/Appstore media app requirements before M4

Open questions

- [x] ~~Release target: Amazon Appstore, at store quality~~
- [x] ~~Server model: one active server at a time; only search spans servers, with "Also on" switching~~
- [ ] Get a Fire TV Stick 4K Select, currently the Vega OS device, for testing
- [ ] Support Plex remote relay connections, or local and direct only?
- [ ] Pricing: free, paid, or a one-time unlock through Vega in-app purchasing?
- [ ] Should each profile remember a default server?

Sources

- React Native for Vega overview
- Focus management on Vega
- Selecting the playback mode (URL vs MSE)
- Supported libraries and services
- Test your app before submitting (Vega)
- Accessibility resources for Vega
- App submission for Vega